The Plaksha Orbital Pipeline Deck !!

CS2011: Foundations of Computer Systems

Instructor: Ankur Nahar

The Plaksha Orbital Pipeline Deck

Far above Earth, the *Plaksha Orbital Computing Deck* is running a next-generation processor that powers simulation pods, navigation displays, sample collection, and autonomous assistants. Everything appears smooth from the outside, but inside the processor a familiar problem keeps appearing: instructions do not always cooperate. Some move forward cleanly, some stuck due to dependencies, and some create stalls that slows the entire flow. Your task in this assignment is to build an applet that lets a user look inside this processor and observe what really happens as instructions travel through the pipeline. The objective is to create a correct, readable, and engaging simulator (applet) that makes pipelining and hazards easy to understand.

Objective

This assignment asks you to design and implement an interactive Pipeline Hazard Simulation Applet (web-based solution, desktop-based solution, or other runnable graphical implementation will also work). The applet should allow a user to enter a short sequence of instructions, choose a pipeline model, and observe how those instructions move through the pipeline cycle by cycle. The applet must make it clear when instructions progress normally, when hazards occur, and how those hazards are resolved. The goal of the assignment is to help you build an understanding of instruction pipelining through implementation and visualization. A good submission will make the internal pipeline behaviour visible in a way that is technically correct and easy to follow.

What You Are Expected to Build

You are required to develop an applet that simulates a **single-issue, in-order pipeline**. At most one instruction may enter the pipeline in a given cycle, and instructions must move through the pipeline in program order. The applet must support user-defined instruction sequences, configurable pipeline structures, cycle-by-cycle execution, and hazard visualization. The focus of the assignment is the logic of pipeline flow. You are not required to simulate the entire behaviour of a real processor. The system does not need to model caches, branch prediction, out-of-order execution, superscalar issue, exceptions, interrupts, or multi-cycle functional units. The simulator should remain compact, understandable, and correct.

Scope of Hazard Simulation

The core hazard that must be supported in this assignment is the *RAW hazard*, or *Read After Write*. This occurs when one instruction needs to read a value that is being produced by an earlier instruction and is not yet available. Your applet must detect this case correctly and must reflect its effect on the pipeline schedule. The simulator is based on a simple in-order pipeline, so *WAR* and *WAW* hazards are not required in the baseline implementation. If you choose to support WAR or WAW, you must clearly explain what architectural assumptions make them possible in your model. These hazards should not be added casually or claimed without justification.

Control and structural hazards are outside the scope of this assignment. You are therefore not required to support branch or jump instructions.

Instruction Input Requirements

Your applet must allow the user to enter instructions dynamically. The user must be able to add instructions, remove instructions, and modify instructions before running the simulation. The total number of instructions in one simulation must be configurable, but the maximum allowed number is 10. This limit is intended to keep the visualization readable and to prevent unnecessary complexity.

At minimum, the applet must support the following instruction types: `ADD`, `SUB`, `LW`, and `SW`. These instructions are sufficient to demonstrate arithmetic dependencies and memory-related behaviour. You may support additional instructions if you wish, but the required functionality must work correctly for these four instructions. The accepted instruction syntax must be clearly defined and used consistently.

Valid examples include:

- ADD R1, R2, R3
- SUB R4, R1, R5
- LW R1, 0(R2)

If your applet validates instruction format and rejects incorrect input, the accepted format must be explained clearly in the report.

Pipeline Configurations to be Supported

The user must be able to choose between two pipeline configurations.

The first configuration is a **4-stage pipeline**. This model must contain the stages **IF**, **ID**, **EX**, and **MEM/WB**. In this pipeline, memory access and write-back are treated as one combined final stage.

The second configuration is a **5-stage pipeline**. This model must contain the stages **IF**, **ID**, **EX**, **MEM**, and **WB**. This is the standard textbook pipeline and must be simulated distinctly from the 4-stage version.

Your applet must support both configurations correctly, and the selected pipeline must affect the resulting execution schedule.

Expected Behaviour of the Applet

The applet must allow the user to enter an instruction sequence, select the pipeline type, and then run the simulation either one cycle at a time or continuously until completion. The user must also be able to reset the simulation to start again from the beginning.

The most important output is the *pipeline execution table*. This table must show time along one dimension and instructions along the other. Each instruction should appear as a row, each cycle should appear as a column, and every populated cell should show the stage occupied by that instruction during that cycle.

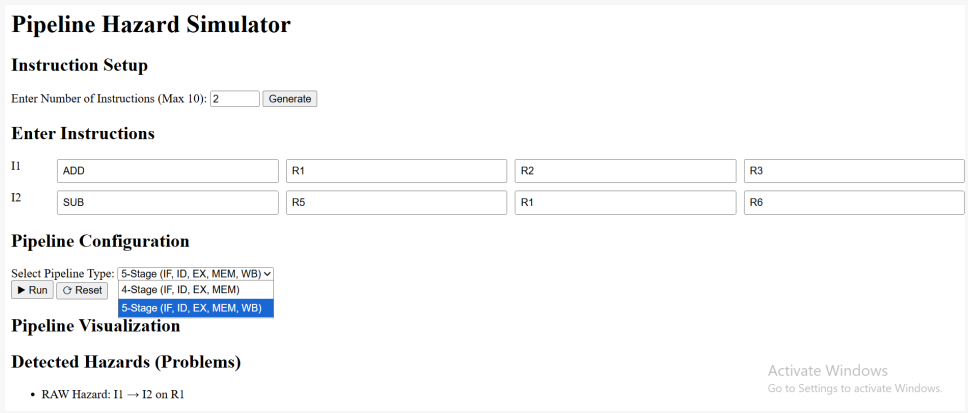
An example table:

Cycle	1	2	3	4	5	6	7
I1	IF	ID	EXE	MEM	WB		
I2		IF	ID	EXE	MEM	WB	
I3			IF	ID	EXE	MEM	WB

The visualization must clearly distinguish normal progress from delayed progress. If a stall is inserted, it must be shown explicitly using a clear label such as `STALL` or `BUBBLE`. Empty cells must not create ambiguity. A user should be able to understand whether an instruction has not yet started, has completed, or has been delayed.

The user interface should be simple, readable, and intuitive. The purpose of this assignment is to make the pipeline understandable, not to bury the simulation under unnecessary interface complexity.

A sample screenshot of the applet



Hazard Detection and Resolution

Your applet must detect *RAW hazards* correctly and must handle them through stall insertion. This is the required baseline behaviour. Whenever a RAW hazard occurs, the applet should make it possible for the user to understand which instruction depends on which earlier instruction and why the pipeline must pause.

You also need to implement *data forwarding* as an enhancement. When forwarding is selected, the applet must clearly show where forwarding is used and how it changes the schedule compared with pure stalling. Forwarding must not be merely claimed; it must be logically implemented and visibly reflected in the simulation.

Correctness Expectations

The applet will be evaluated primarily on correctness and consistency. If two instructions are independent, the simulator should allow them to proceed without unnecessary stalls. If one instruction depends on a value that is not yet available, the applet must delay it according to the rules of the selected pipeline.

Your report must clearly explain the assumptions used by your simulator. For instance, if a register result becomes available only after the *WB* stage, that assumption must be stated. If forwarding is implemented, the exact forwarding rule must also be described. The simulation output must match your stated assumptions consistently throughout the system.

What You Must Submit

Your submission (zip/rar) must contain three components. The first component is the source code of your applet. It must include all necessary files required to run or build the project. The second component is a working runnable version of the project. This may be a web-based applet, a desktop application, or another executable form, as long as it can be evaluated clearly and without confusion. The third component is a report in PDF format. The report must explain the design of your applet, the assumptions used in the simulation, the instruction format supported by the system, and the logic used to detect and resolve hazards. It must also explain how to run the applet, include screenshots of the interface and simulation output, and provide sample test cases demonstrating correct behaviour.

Testing Expectations

Your report should include enough testing evidence to show that the simulator is functioning correctly. At minimum, it should contain one instruction sequence with no dependencies, one sequence with a simple arithmetic RAW hazard,

and one sequence with a load-use dependency. If forwarding is implemented, at least one test case should show how the schedule changes when forwarding is enabled.

Implementation Freedom

You may use any language, framework, or toolkit to implement the applet. A web-based implementation is acceptable. A desktop-based implementation is also acceptable. The choice of technology is open, but the final system must be executable and understandable. The interface may be themed creatively if you wish.

Evaluation Scheme

The assignment will be graded according to the following rubric.

Criterion	**Marks**
Instruction input handling, editing support, and correctness of accepted format	2.5
Support for both 4-stage and 5-stage pipeline configurations	2.5
Pipeline visualization and clarity of cycle-by-cycle execution	8.0
Quality of user interaction, including step execution, auto-run, and reset	3.0
Correct RAW hazard detection and stall-based resolution	3.0
Report quality, code organization, and clarity of execution instructions	1.0
Total	**20.0**

The strongest submissions will be the ones that make the invisible behaviour of the pipeline visible in a simple, convincing, and technically correct way.

Important Policies

- This assignment may be completed individually or in a team of up to three students. The minimum team size is one and the maximum team members size is three.
- Discussion across teams is encouraged for learning purposes, but every team must independently design and implement its own solution. Any form of code copying will result in zero marks. No two implementations should be identical in design, structure, or code.
- You may use AI tools to understand concepts, clarify syntax, or explore ideas. However, the final program, logic, and implementation must be submitted with proper understanding.
- The final deadline for submission is **26 April 2026 at 11:59 PM**.
- Late submission penalties will be applied strictly. A delay of even one minute beyond the deadline will be treated as late submission. A penalty of **1 mark** will be applied for the first 12 hours of delay, **2 marks** for the next 12 hours, and **3 marks per day** for each additional day thereafter.

Closing Note

Imagine that each instruction entering your applet is a transport capsule moving through the Plaksha orbital processor deck. Most glide forward smoothly. Some must pause because the value they need has not arrived yet. Your simulator should let the user see that invisible choreography clearly. If a student can look at your applet and finally say, "Now I understand what a pipeline hazard actually does," then you have built exactly the right system.

Build it clearly. Simulate it correctly. Make the pipeline visible.